

Architecture Logicielle

Conception de l'architecture logicielle et des interfaces entre composants

Ce rapport présente le pattern architectural retenu (monolithe modulaire Next.js 14), les diagrammes C4 niveaux 1 et 2, les diagrammes de séquence clés, les interfaces entre composants et les mécanismes de sécurité et de scalabilité.

Projet	CareerAI — Plateforme SaaS de recherche d'emploi
Stack	Next.js 14 · Supabase · Groq IA · Resend
URL	https://www.careerai.live
GitHub	github.com/ARTHURNJOUONANG/Projet_Chat_board

1. Pattern architectural retenu

■ ■ ■ **Monolithe modulaire full-stack Next.js 14** — Ce pattern évite la dispersion de la logique métier dans les composants UI tout en gardant un seul artefact déployable. Il est adapté à la taille actuelle de l'équipe et facilite les itérations rapides.

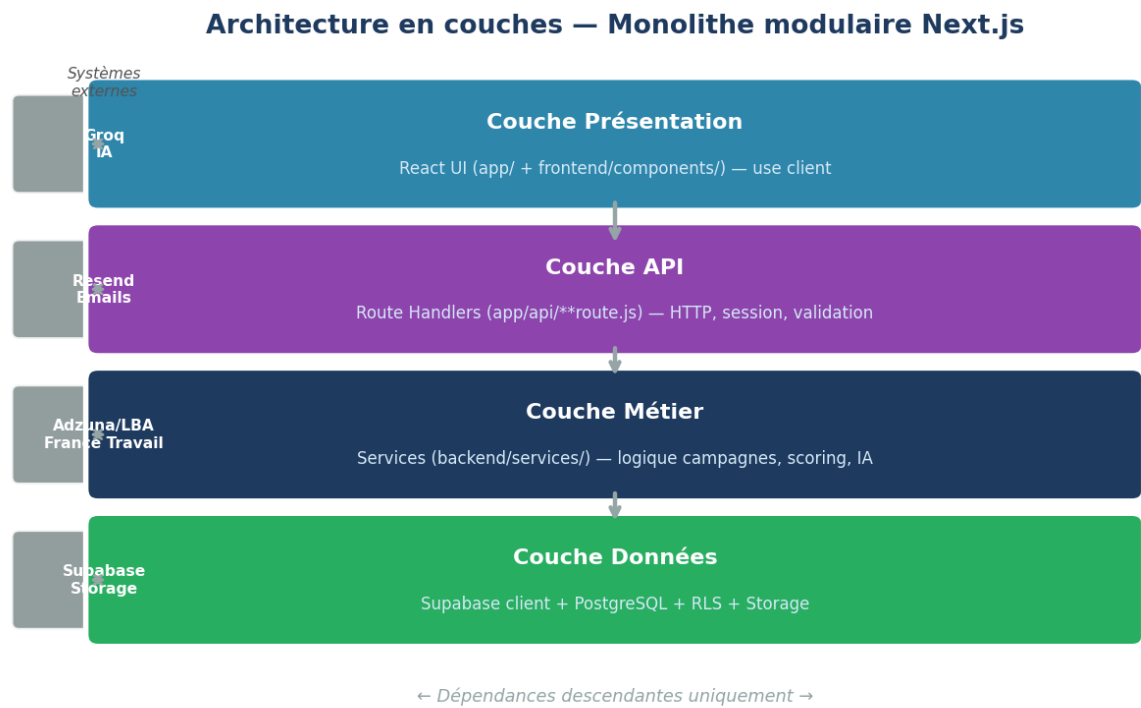


Figure 1 — Architecture en couches du monolithe modulaire

Couche	Répertoires	Responsabilité
Présentation	app/ + frontend/components/	Pages React, composants UI (use client)
API	app/api/**/route.js	Point d'entrée HTTP, validation session, orchestration
Métier	backend/services/ + app/lib/	Logique réutilisable, appels externes, règles campagne
Données	Supabase client + PostgreSQL	Persistance, RLS, Storage documents

2. Diagramme C4 — Niveau 1 : Contexte

Le diagramme de contexte positionne CareerAI au centre de son écosystème, en identifiant les acteurs humains et les systèmes externes avec lesquels il interagit.

C4 — Niveau 1 : Contexte système CareerAI

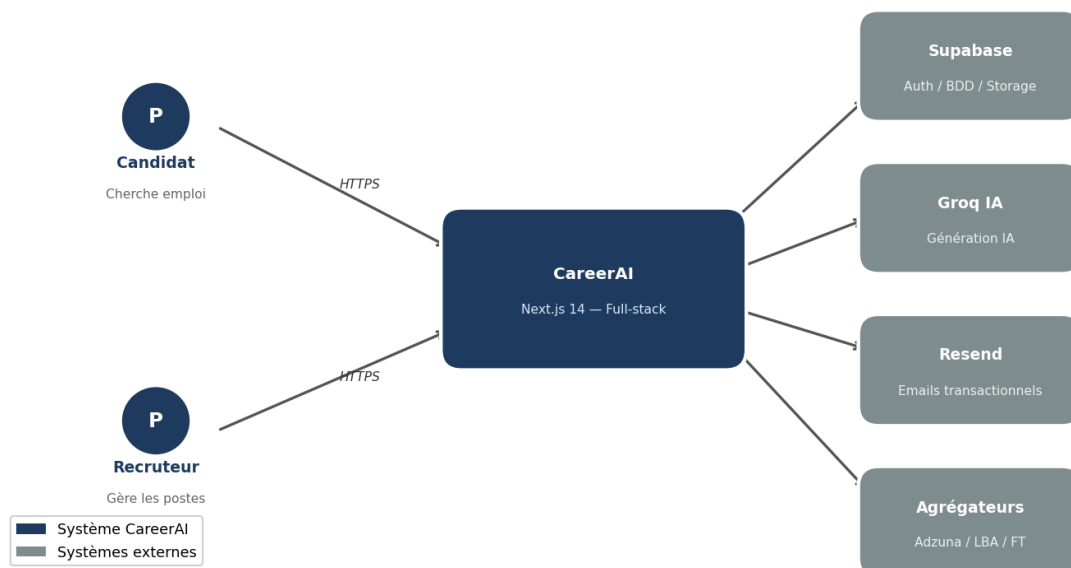


Figure 2 — C4 Niveau 1 : Contexte système CareerAI

3. Diagramme C4 — Niveau 2 : Conteneurs

Le diagramme de conteneurs détaille les blocs déployables et leurs interactions. Le navigateur (ou APK Android) communique avec l'application Next.js, qui s'appuie sur Supabase pour la persistance et sur des APIs externes pour l'IA, les emails et les offres.

C4 — Niveau 2 : Conteneurs

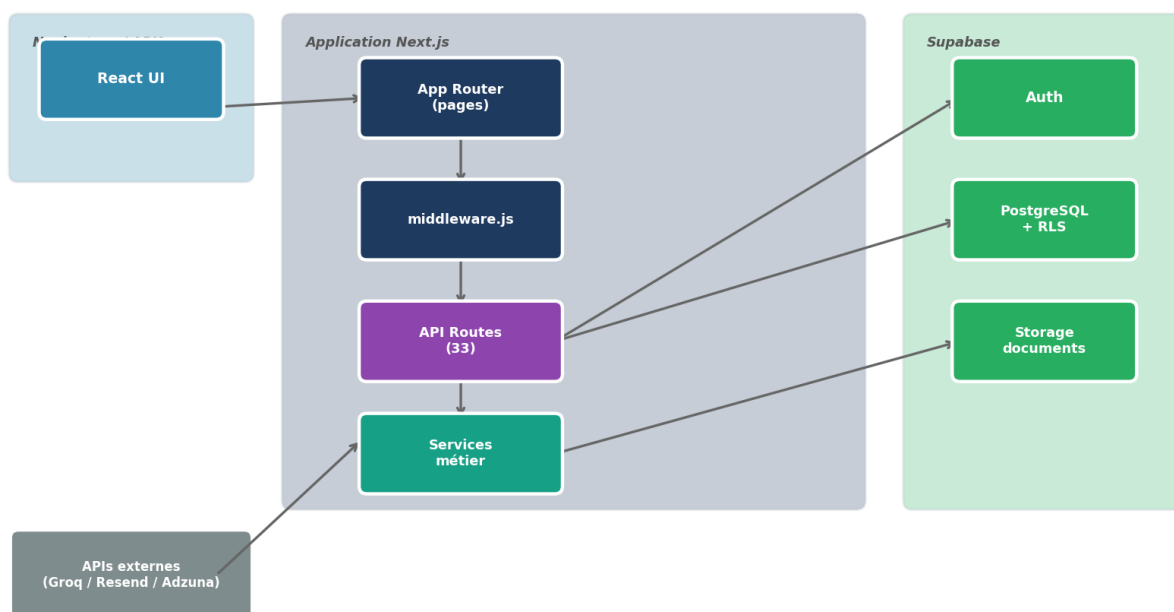


Figure 3 — C4 Niveau 2 : Conteneurs

4. Diagrammes de séquence

4.1 Authentification

À l'inscription, Supabase Auth crée la session. Le middleware Next.js intercepte chaque requête pour vérifier la validité du cookie et rediriger si nécessaire.

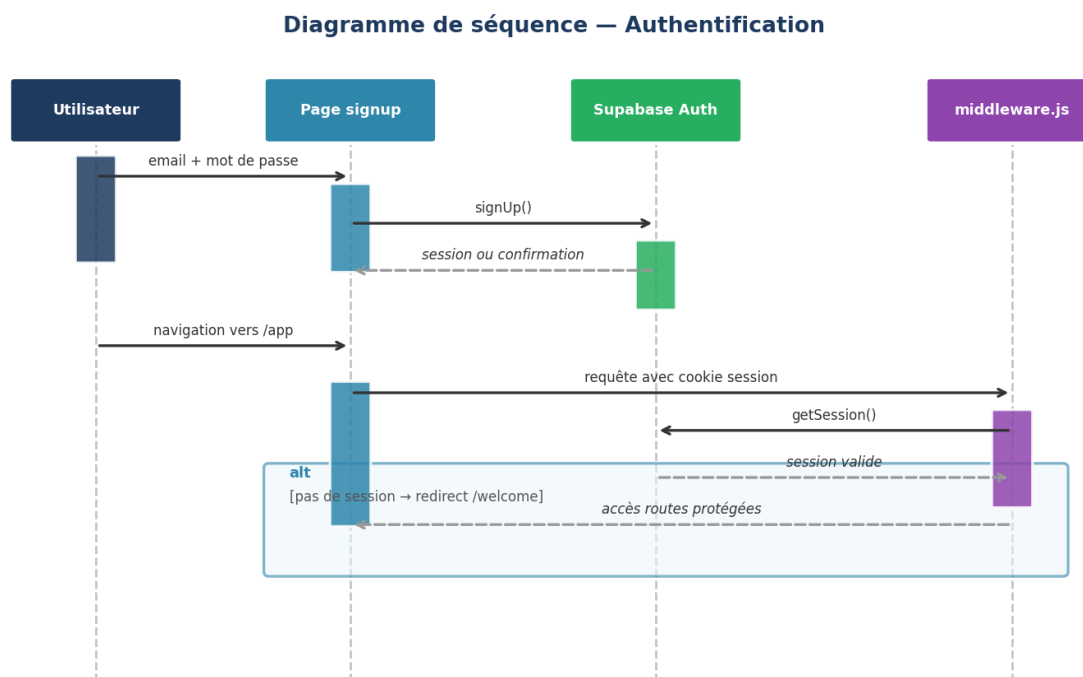


Figure 4 — Séquence : Authentification

4.2 Exécution campagne (run-now)

Le candidat déclenche manuellement une campagne. La route API vérifie la session, délègue au CampaignService qui orchestre la récupération des offres, le matching, l'envoi des candidatures et l'enregistrement des résultats.

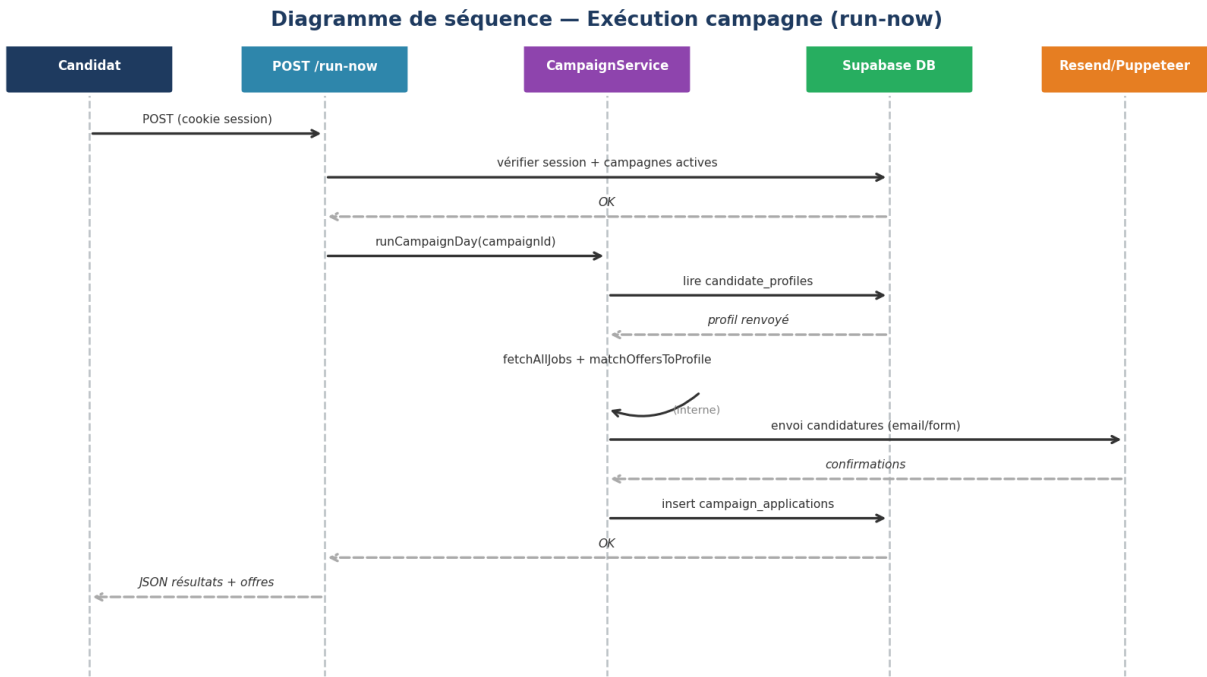


Figure 5 — Séquence : Exécution campagne run-now

5. Interfaces entre composants

5.1 UI → API (REST interne)

Méthode	Endpoint	Rôle
GET	/api/campaigns	Liste des campagnes de l'utilisateur
POST	/api/campaigns	Créer une campagne
POST	/api/campaigns/run-now	Exécution immédiate d'une campagne
POST	/api/recruiter/candidates	Ajouter un candidat (multipart CV)
POST	/api/recruiter/quizzes/{id}/send	Envoyer un quiz par email
GET	/api/recruiter/rankings	Classements par poste
GET/POST	/api/candidate/quiz/{token}	Accès quiz public par token

5.2 API → Services métier

Service	Méthode principale	Description
CampaignService	runCampaignDay(campaignId)	Orchestration complète d'une campagne
RankingService	computeRankings(jobId)	Calcul des scores et classements recruteur
AIService	generateContent(prompt)	Appels Groq pour génération IA

6. Sécurité et scalabilité

Aspect	Approche
Auth session	Cookies Supabase vérifiés par middleware à chaque requête
RLS	Politiques PostgreSQL par user_id / recruter_id
Secrets serveur	Variables .env.local uniquement côté Node.js
Cron	Header Authorization: Bearer CRON_SECRET
API stateless	Pas de session serveur Next — scaling horizontal possible (Vercel/Render)
Plafonds métier	max_applications_per_day, durée campagne contrôlés en base
Traitements longs	maxDuration sur routes campagne, exécution asynchrone via cron
Cache	Pas de cache distribué V1 — évolution Redis si volume